

Design Document minired: A Single-Threaded In-Memory Key-Value Server Implementing the Redis Wire Protocol

Course: Advanced Database Management — Computer Science, 4th Semester Project
Project: Project 02 — BSCS24135-BSCS24031

1. Introduction

Redis is among the most widely deployed pieces of infrastructure software in the world, yet its architecture is almost the inverse of what a systems design course might suggest. There is no thread pool, no connection pool, no query planner, no write-ahead log, and no two-phase commit. Instead, Redis achieves extraordinary performance through disciplined simplicity: keep everything in memory, run one thread, use hand-tuned data structures, and push persistence to a background process that never touches the critical path.

Building a Redis-compatible server from scratch forces a confrontation with each of these design decisions. Why is a single-threaded model faster than a multi-threaded one for this workload? Why does the RESP protocol use length-prefixed strings rather than null termination? Why does a skip list outperform a balanced binary search tree in practice despite identical asymptotic complexity? These are not rhetorical questions — they have concrete, measurable answers that only become apparent when you implement the alternatives yourself.

minired answers all of them. The remainder of this document describes each major component of the implementation in detail: the event loop, the protocol layer, the key-value dictionary, each of the four data type implementations, the expiration mechanism, and the persistence subsystem. The final section discusses benchmark results and what they reveal about the bottlenecks in the design.

2. Event Loop Architecture

2.1 The Case for Single-Threading

The most counterintuitive aspect of the minired architecture is the complete absence of threads in the command execution path. A typical server accepts a connection, spawns a thread to handle it, and relies on the operating system scheduler to multiplex among clients. minired does none of this. One thread handles every accepted connection, reads every request, executes every command, and writes every reply.

The justification is rooted in the nature of the workload. An in-memory key-value server spends almost no time doing CPU work. A GET command performs a hash table lookup and writes perhaps thirty bytes to an output buffer. On modern hardware this takes on the order of hundreds of nanoseconds. The network round trip to deliver the reply takes hundreds of microseconds. The server is idle, from the CPU's perspective, for more than ninety-nine percent of each request's lifetime.

A multi-threaded server does not improve CPU utilization in this regime — there is no CPU work to parallelize. What it does introduce is overhead: thread creation and destruction, context switching, cache line invalidation when multiple threads contend for the same hash table buckets, and the cognitive and correctness burden of lock-based synchronization. Redis's insight, which minired reproduces, is that all of this overhead can be eliminated entirely by adopting a model where the server is always doing exactly one thing at a time.

The discipline this imposes is strict: no operation in the server may block. Every function that touches a client connection must return immediately or not be called at all. The operating system's I/O multiplexing interface — `epoll` on Linux, `kqueue` on macOS — provides the mechanism by which a single thread can monitor thousands of sockets simultaneously and be notified when any of them has work to do.

2.2 Cross-Platform I/O Multiplexing

minired targets both Linux and macOS from a single codebase. The two platforms provide different system calls for I/O multiplexing: Linux provides `epoll`, introduced in kernel 2.5.44, and macOS provides `kqueue`, derived from FreeBSD. Both interfaces share the same conceptual model — register a set of file descriptors, wait for one or more of them to become ready, process the ready ones, repeat — but differ in their API surface.

The implementation wraps both behind a thin platform abstraction. A compile-time preprocessor guard selects the correct implementation:

```
#ifdef linux #include <sys/epoll.h> #define USE_EPOLL 1 #else #include <sys/event.h>
#define USE_EPOLL 0 #endif
```

The three operations that interact with the platform — adding a file descriptor, modifying its interest set, and waiting for events — are each implemented twice and selected at compile time. The rest of the server, from the accept loop to the command dispatcher, is entirely portable.

The choice of `epoll` and `kqueue` over the older `select` and `poll` interfaces is not merely aesthetic. `select` imposes a hard limit of 1024 file descriptors on most systems and requires the application to rebuild its interest set on every call. `poll` removes the 1024 limit but requires a linear scan of all registered descriptors on every call, making it $O(n)$ in the number of connections regardless of how many are actually ready. `epoll` and `kqueue` are $O(k)$ where k is the number of ready events, which is typically a small constant. For a server handling thousands of idle connections with a few active ones, this difference is decisive.

2.3 Non-Blocking Sockets and the EAGAIN Contract

Every socket the server creates — the listening socket and every accepted client socket — is placed into non-blocking mode using:

```
fcntl(fd, F_SETFL, fcntl(fd, F_GETFL, 0) | O_NONBLOCK);
```

In non-blocking mode, `recv` and `send` never sleep. If `recv` is called and there is no data available, it returns -1 with `errno` set to `EAGAIN` rather than blocking until data arrives. If `send` is called and the kernel's send buffer is full, it returns -1 with `EAGAIN` rather than

blocking until space becomes available. The server interprets EAGAIN as "come back later" and returns control to the event loop.

This is the mechanism by which a single thread can interleave work across many clients. When client A's socket has no data, `recv` returns immediately and the thread moves on to client B. When client B's send buffer is full, `send` returns immediately and the thread moves on to client C. No client monopolizes the thread.

2.4 Per-Client State and Buffer Management

Each accepted connection is represented by a `Client` struct containing four fields: the socket file descriptor, a read buffer, a write buffer, and a closing flag.

```
struct Client { int fd; std::string read_buf; std::string write_buf; bool closing; };
```

The read buffer accumulates bytes from `recv` calls. The RESP parser operates on this buffer and consumes bytes as it successfully parses complete commands. Bytes belonging to a partially received command are left in place; they will be joined by the next `recv` and re-parsed. This is the mechanism by which the incremental parser handles TCP fragmentation.

The write buffer accumulates reply bytes produced by command handlers. The event loop flushes this buffer to the socket when the socket is writable. The key discipline here is that `EPOLLOUT` — the writable event — is registered only when the write buffer is non-empty. A socket is almost always writable; registering for writable events unconditionally would cause the event loop to wake up on every iteration for every connected client even when there is nothing to send. This is a classic performance mistake that the implementation explicitly avoids.

2.5 The Event Loop Body

The event loop runs until the process is killed. Each iteration calls `epoll_wait` or `kevent` with no timeout, blocking until at least one file descriptor is ready. For each ready descriptor, the loop checks whether it is the listening socket (a new connection is arriving) or a client socket (data is available or the send buffer has drained). The complete loop body is:

```
for each ready event: if fd == listen_fd: accept new connection set O_NONBLOCK register
with epoll/kqueue for EPOLLIN allocate Client struct else: if readable: recv into client-
>read_buf parse complete commands dispatch each command if client->closing: close and
free if writable and client exists: send from client->write_buf if write_buf empty: deregister
EPOLLOUT
```

The check "if client exists" before handling the writable event is necessary because the readable handler may have closed the client. Without this guard, a use-after-free occurs on the client pointer.

3. The RESP Protocol

3.1 Protocol Overview

The Redis Serialization Protocol is the wire format used for all communication between redis-cli and the server. It was designed with three goals: simplicity of implementation, human readability for debugging, and binary safety. The last property is the most important for a key-value store: values may contain arbitrary bytes including null bytes and newlines, and the protocol must be able to represent them without ambiguity.

RESP achieves binary safety through length-prefixed encoding. Rather than delimiting values with a special byte that must be escaped when it appears in data, RESP states the byte count of each value before transmitting it. The receiver reads exactly that many bytes, regardless of their content.

3.2 The Five RESP Types

Every value transmitted over the wire is prefixed with a single type byte:

Simple String (+): Used for short success responses. The format is `+OK\r\n`. The value is everything between the type byte and the first CRLF. Simple strings cannot contain embedded CRLFs; they are not binary-safe and are used only for short status messages.

Error (-): Identical in structure to a simple string but indicates failure. The convention is that the first word of the error message is an error code in uppercase: `-ERR unknown command\r\n`, `-WRONGTYPE Operation against a key holding the wrong kind of value\r\n`. redis-cli renders error replies in red.

Integer (:): A signed 64-bit integer rendered in ASCII decimal followed by CRLF: `:42\r\n`. Used for counts, boolean values (0 and 1), and TTL values. The integer type is never used for values stored under keys, only for metadata returned by commands.

Bulk String (\$): A length-prefixed binary-safe string. The format is `$5\r\nhello\r\n` — the length, a CRLF, exactly that many bytes, and another CRLF. The second CRLF is a terminator, not part of the value. A null bulk string, `$-1\r\n`, represents the absence of a value and is returned by GET on a missing key.

Array (*): A length-prefixed sequence of any RESP values: `*3\r\n....`. Every command from the client arrives as an array of bulk strings. The first element is the command name; subsequent elements are arguments. This uniformity means the server parser only needs to handle one top-level case.

3.3 Incremental Parsing

The most consequential design decision in the protocol layer is the handling of partial input. TCP is a stream protocol. There is no guarantee that a single `recv` call will return exactly one complete RESP message. A call might return half a command if the network delayed the second half, or it might return three commands if they were pipelined by the client. The parser must handle both cases correctly.

The parser function has the following signature:

```
RespResult resp_parse(const char* buf, size_t buflen);
```

It returns a struct containing the parse status (OK, INCOMPLETE, or ERROR), the parsed argument vector if successful, and the number of bytes consumed. Critically, it does not modify the buffer — it only reports how many bytes it used. The caller is responsible for advancing the buffer's read position.

The INCOMPLETE status is returned whenever the buffer contains a valid but unfinished RESP message: the array header has been read but not all bulk strings have arrived, or a bulk string's length has been read but not all its bytes. When INCOMPLETE is returned, the caller retains all bytes in the buffer and waits for the next `recv`.

This design was validated by a unit test that feeds the parser exactly one byte at a time. The test sends a complete PING command fourteen bytes long and verifies that the parser returns INCOMPLETE for each of the first thirteen calls and OK on the fourteenth. This is the hardest case the parser faces in practice; if it handles single-byte delivery correctly, it handles all fragmentation patterns correctly.

3.4 Inline Command Support

redis-cli sends all commands as RESP arrays. However, tools like telnet and netcat send commands as plain text lines. minired supports inline commands — a line of space-separated tokens followed by CRLF — as a fallback. When the first byte of the buffer is not `*`, the parser falls back to splitting the line on spaces. This makes the server accessible for debugging without a full RESP client.

4. The Key-Value Dictionary

4.1 Design Requirements

The top-level dictionary is the most performance-critical data structure in the server. Every command touches it — GET and SET obviously, but also TTL (to check expiry), TYPE (to inspect the value's tag), and KEYS (to iterate all entries). The dictionary must support $O(1)$ average-case lookup, insertion, and deletion, and must grow dynamically as keys are added.

4.2 Hash Table Implementation

The dictionary is implemented as a chaining hash table. Each bucket is the head of a singly linked list of entries. Each entry stores a key (as an sds string), a value (as an Obj pointer), and a pointer to the next entry in the chain.

The hash function is FNV-1a (Fowler-Noll-Vo), a well-known non-cryptographic hash function with excellent distribution properties for short strings:

```
static size_t dict_hash(const char* key, size_t klen) { size_t hash =
14695981039346656037ULL; for (size_t i = 0; i < klen; i++) { hash ^= (unsigned
char)key[i]; hash *= 1099511628211ULL; } return hash; }
```

FNV-1a processes one byte at a time with a single XOR and multiply per byte. It is not the fastest possible hash function — MurmurHash3 and xxHash are faster on modern hardware

due to vectorization — but it is simple to implement correctly and produces minimal collisions for typical Redis key distributions.

The table begins with sixteen buckets and doubles when the load factor (count / size) exceeds 0.75. Doubling on resize ensures that each key is rehashed at most a logarithmic number of times over its lifetime. The resize operation allocates a new bucket array, iterates all existing entries, relinks each into its new bucket, and frees the old array. This is an $O(n)$ operation that blocks the event loop for the duration; for the key counts typical in this project, the pause is immeasurable.

4.3 The Obj Tagged Union

Each dictionary value is an Obj struct containing a type tag, an expiration timestamp, and a union of pointers to the four possible value types:

```
struct Obj { ObjType type; int64_t expire_at_ms; union { char* str; List* list; HTable* hash; ZSet* zset; }; };
```

The type tag is checked before every access to the union. Commands that receive a key of the wrong type return a `WRONGTYPE` error, matching Redis's behavior exactly. The `expire_at_ms` field stores the absolute expiration time in milliseconds since the Unix epoch; a value of zero means no expiration is set.

5. Data Type Implementations

5.1 Strings and the sds Type

Redis strings are binary-safe: they may contain any byte sequence including embedded null bytes. The standard C `char*` type is inadequate because it uses null termination to delimit strings, making it impossible to represent a string containing a null byte. minired implements the sds (Simple Dynamic String) type used by Redis itself.

An sds string is a `char*` pointer that points not to the beginning of a heap allocation but to the middle of one. The bytes immediately preceding the pointer contain a header with the string's current length and allocated capacity:

```
struct sds_header { size_t len; size_t cap; char data[]; };
```

```
typedef char* sds;
```

The pointer returned to callers is `header->data`, not `header` itself. This means an sds can be passed to any function expecting `const char*` — `strlen`, `strcmp`, `printf` — and these functions will work correctly as long as the string contains no embedded nulls. But length queries use `sds_get_header(s)->len`, which reads backward from the pointer to find the header, and they return the true binary length regardless of embedded content.

When an append operation would exceed the current capacity, the allocation is doubled. This amortizes the cost of repeated appends to $O(1)$ per byte appended, matching the behavior of `std::string` and other dynamic array types.

The negative-offset trick is the most unusual aspect of the design. Obtaining the header from an sds pointer requires subtracting the header size from the pointer value:

```
inline sds_header* sds_get_header(sds s) { return reinterpret_cast<sds_header*>(s -
sizeof(sds_header)); }
```

This works because the header is allocated as a single contiguous block; the data array immediately follows the len and cap fields in memory. The pointer arithmetic is well-defined in C++ because we are computing an offset within a single allocation.

5.2 Lists

The List type is a doubly linked list of sds values. Each node contains a value, a pointer to the previous node, and a pointer to the next node. The list itself contains head and tail pointers and a length counter.

```
struct ListNode { sds value; ListNode* prev; ListNode* next; };
```

```
struct List { ListNode* head; ListNode* tail; size_t len; };
```

LPUSH and RPUSH are $O(1)$ — they allocate a new node and update two pointers. LPOP and RPOP are also $O(1)$ — they unlink the head or tail node and free it. LLEN is $O(1)$ because the length is maintained incrementally.

LRANGE and LINDEX require traversal and are $O(n)$ in the worst case. LINDEX optimizes slightly by walking from whichever end is closer to the requested index. LRANGE with negative indices requires index normalization: index -1 refers to the last element, -2 to the second-to-last, and so on. Normalization adds the list's length to any negative index before use.

The implementation does not use Redis's ziplist optimization, which compresses small lists into a contiguous memory region to reduce pointer overhead. For the purposes of this project, the linked list representation is sufficient and considerably simpler to implement correctly.

5.3 Hashes

A Redis hash is a collection of field-value pairs stored under a single top-level key. It is conceptually a dictionary nested inside the top-level dictionary. The implementation reuses the Dict data structure directly, wrapping it in an HTable struct:

```
struct HTable { Dict* d; };
```

All hash commands — HSET, HGET, HDEL, HEXISTS, HLEN, HKEYS, HVALS, HGETALL — delegate to the corresponding dict operations. This reuse is possible because the hash's fields and the top-level dictionary's keys share the same representation: both are sds strings, and both use the same FNV-1a hash function and the same chaining collision resolution.

HGETALL requires iterating all entries in the hash's dictionary. The dict_each function accepts a callback and a user-data pointer, invoking the callback for each entry. HGETALL's

callback accumulates field-value pairs into a pair of sds arrays that are then serialized as a flat RESP array with alternating field and value elements.

5.4 Sorted Sets and the Skip List

The sorted set is the most algorithmically interesting data type in the project. It maintains a collection of (member, score) pairs, where members are unique strings and scores are doubles, and supports efficient retrieval both by member identity and by score order.

Redis implements sorted sets as a combination of two structures: a hash table mapping members to scores for $O(1)$ membership and score queries, and an ordered structure for $O(\log n)$ rank-based queries. minired follows the same design, using the existing Dict for the hash table and a skip list for the ordered structure.

5.4.1 Skip List Theory

A skip list is a randomized data structure that provides the same asymptotic guarantees as a balanced binary search tree — $O(\log n)$ expected time for search, insertion, and deletion — without the complex rotation logic required to maintain balance. It was introduced by Pugh in 1990 and has since become a standard alternative to balanced trees in systems where implementation simplicity is valued.

The structure is a sorted linked list augmented with additional forward pointers at multiple levels. Level 0 contains all nodes in sorted order. Level 1 contains roughly half the nodes, level 2 roughly a quarter, and so on. A node's level — the number of forward pointers it carries — is chosen randomly at insertion time: with probability $p = 0.5$, a new node has at least two levels; with probability $p^2 = 0.25$, it has at least three; and so on. The maximum level is capped at 32, which provides $O(\log n)$ behavior for up to 2^{32} elements.

Search begins at the highest occupied level of the header node and walks forward until the next node either overshoots the target or does not exist, then drops down one level and continues. This skipping behavior gives the $O(\log n)$ expected complexity: at each level, the search skips over approximately half the remaining nodes.

5.4.2 Skip List Implementation

The implementation follows Pugh's original pseudocode closely. The header node is a sentinel with an empty member, the minimum possible score, and forward pointers at all 32 levels. All traversals begin at the header.

Insert requires two passes. The first pass builds an update array recording the rightmost node at each level whose successor would follow the new node. The second pass allocates the new node with a random level, then links it into the list at each of its levels by splicing it between `update[i]` and `update[i]->forward[i]`.

Delete follows the same two-pass structure. The first pass builds the update array; the second unlinks the target node from each level. After deletion the level of the skip list is reduced if the top levels have become empty.

A subtle correctness issue arises when a member is reinserted with a new score. Because the skip list is ordered by score, a node with an updated score may belong at a completely different position. The simplest correct implementation removes the old node entirely before inserting the new one, which the `zset_add` function does:

```
bool zset_add(ZSet* z, const char* member, size_t mlen, double score) { double old_score; if (ht_get_score(z->ht, member, mlen, &old_score)) { sl_delete(z->sl, member, mlen); } sl_insert(z->sl, member, mlen, score); ht_set_score(z->ht, member, mlen, score); return true; }
```

The hash table is updated atomically with the skip list, ensuring that the two structures never disagree about a member's score.

5.4.3 Rank and Range Queries

`ZRANK` returns the zero-based rank of a member in the sorted order. The implementation walks level 0 of the skip list from the header, counting nodes until the target member is found. This is $O(n)$ in the worst case but $O(\log n)$ on average due to the skip list's probabilistic structure at higher levels.

`ZRANGE` retrieves elements by rank range, supporting both positive and negative indices. Negative indices are normalized by adding the set's cardinality before use. The traversal walks to the start rank at level 0, then collects elements until the end rank is reached.

6. Key Expiration

6.1 Lazy Expiration

Redis supports per-key time-to-live (TTL) values. A key with an active TTL is automatically deleted when its TTL elapses. `miniredis` implements expiration using lazy deletion: the expiry is not checked on a schedule, but on every access to the key. Before any command reads or writes a key, it calls `check_expiry`:

```
bool check_expiry(Dict* d, const char* key, size_t klen) { Obj* obj = dict_get(d, key, klen); if (!obj) return false; if (obj->expire_at_ms == 0) return false; if (now_ms() < obj->expire_at_ms) return false; dict_del(d, key, klen); return true; }
```

If the key exists, has an expiration set, and the current time exceeds the expiration timestamp, the key is deleted and `true` is returned. The calling command then proceeds as if the key had never existed — for example, `GET` returns a null bulk string, `EXISTS` returns 0, and `SET` creates a new key.

The expiration timestamp is stored as an absolute value in milliseconds since the Unix epoch, computed using `std::chrono::system_clock`:

```
int64_t now_ms() { using namespace std::chrono; return duration_cast<milliseconds>(system_clock::now().time_since_epoch()).count(); }
```

`system_clock` was chosen over `steady_clock` because `system_clock` represents wall-clock time and is consistent across multiple calls to the process, including after process restart when comparing stored timestamps against the current time. `steady_clock` is monotonic but not anchored to any external reference, making it unsuitable for timestamps that must survive across process lifetimes.

6.2 EXPIRE, TTL, and PERSIST

`EXPIRE` sets a key's expiration by computing `now_ms() + seconds * 1000` and storing the result in `obj->expire_at_ms`. It returns 1 if the key exists and 0 otherwise.

`TTL` returns the remaining time-to-live in seconds. It computes `obj->expire_at_ms - now_ms()` and divides by 1000, rounding up to ensure that a key with one millisecond remaining reports one second rather than zero. A return value of -1 indicates no expiration is set; -2 indicates the key does not exist.

`PERSIST` removes a key's expiration by setting `obj->expire_at_ms = 0`. It returns 1 if an expiration was removed and 0 if the key had no expiration or did not exist.

6.3 Expiry on Snapshot Load

When a snapshot is loaded at startup, each key's `expire_at_ms` value is compared against the current time. Keys that expired while the server was offline are silently discarded during loading. This ensures that the server does not resurrect expired keys after a restart, matching Redis's behavior exactly.

7. Persistence: Fork-Based Snapshots

7.1 The Persistence Problem

An in-memory server that stores all data in RAM loses everything when the process exits. For a cache this is often acceptable — the data can be reconstructed from the source of truth. But for many use cases, including session stores, leaderboards, and task queues, losing all data on restart is unacceptable. Persistence must be provided.

The challenge is that persistence requires disk I/O, and disk I/O is orders of magnitude slower than memory I/O. A naive implementation that writes every write command to disk before acknowledging it to the client would reduce throughput from hundreds of thousands of operations per second to the hundreds or low thousands. This defeats the entire purpose of an in-memory store.

Redis's solution, which minired reproduces, is to decouple persistence entirely from the request path. Snapshots are written asynchronously by a child process that never interferes with the parent's ability to serve requests.

7.2 The Fork Trick

The `SAVE` and `BGSAVE` commands both begin by calling `fork()`. This is the key insight of Redis's persistence design. `fork()` on Linux and macOS creates a child process that is an exact

copy of the parent, sharing all memory through a copy-on-write mechanism: physical memory pages are shared between parent and child until one of them writes to a page, at which point the operating system makes a private copy for the writing process.

At the moment of the fork, the child holds a complete, consistent snapshot of the key-value store. It does not matter that the parent continues to process writes — those writes modify the parent's pages, which are copied on write, leaving the child's view frozen at the moment of the fork. The child can therefore iterate the dictionary and write every key to disk without taking any locks, without blocking the parent, and without seeing any of the parent's subsequent modifications.

The child process's work is straightforward: open a temporary file, write the header, iterate the dictionary via `dict_each`, write each entry, write the CRC64 checksum, close the file, and rename the temporary file into place. The rename is atomic at the filesystem level, ensuring that a reader never sees a partially written snapshot file.

SAVE waits for the child with `waitpid`, blocking the event loop until the snapshot is complete, then replies OK. BGSAVE does not wait — it replies immediately and the parent's `SIGCHLD` handler reaps the child when it exits. The `SIGCHLD` handler also logs whether the save succeeded.

7.3 Snapshot File Format

The snapshot format is a simple binary format defined by the project specification. It begins with a four-byte magic number ('MRDB'), a four-byte version number (currently 1), and an eight-byte key count. Each key is then serialized as a sequence of fields:

[type: uint8] [expire_at_ms: int64] [key_len: uint32][key_bytes: N] [type-specific body]

The type-specific bodies are:

String: [len: uint32][bytes: N] List: [count: uint32] followed by count blobs [len: uint32][bytes: N] Hash: [count: uint32] followed by count field-value pairs, each as two blobs
Sorted set: [count: uint32] followed by count score-member pairs [score: double][len: uint32][bytes: N]

The file ends with an eight-byte CRC64 checksum computed over all preceding bytes. The CRC is verified on load; if it does not match, the server refuses to start and reports a corruption error.

All multi-byte integers are written in native byte order. This means the snapshot format is not portable across architectures with different endianness, which is noted as a known limitation. For a production system, big-endian or little-endian normalization would be appropriate.

7.4 CRC64 Correctness Guarantee

The CRC64 implementation uses the polynomial 0xad93d23594c935a9, a 64-bit variant of the CRC standard. The checksum is computed incrementally as bytes are written, using a 256-entry lookup table generated at initialization time. The same computation is performed

on the read path; if the stored and computed checksums differ, even a single flipped bit will be detected with overwhelming probability.

The CRC provides protection against storage media corruption, incomplete writes interrupted by power failure, and accidental truncation of the file. It does not provide protection against deliberate tampering; a cryptographic hash would be required for that purpose. For the intended use case — detecting accidental corruption — CRC64 is the appropriate tool.

8. Benchmark Results and Analysis

8.1 Methodology

Benchmarks were conducted using the redis-benchmark tool that ships with the Redis distribution. This tool opens a configurable number of concurrent connections to the server, fires a large number of pipelined requests through each connection, and reports throughput and latency statistics.

All benchmarks were run on a MacBook Pro with Apple M2 processor. The server ran on port 6380. The benchmark tool ran on the same machine to eliminate network latency from the measurements.

8.2 Results

Operation	Requests	Clients	Throughput	p50	p95	p99	Max
PING	100,000	50	206,611 ops/sec	0.127ms	0.159ms	0.399ms	0.775ms
SET	100,000	50	214,132 ops/sec	0.127ms	0.159ms	0.175ms	0.303ms
GET	100,000	50	208,768 ops/sec	0.127ms	0.143ms	0.151ms	0.607ms
LPUSH	50,000	50	208,333 ops/sec	0.135ms	0.143ms	0.151ms	0.175ms
LPOP	50,000	50	209,205 ops/sec	0.135ms	0.143ms	0.151ms	0.455ms
HSET	50,000	50	210,970 ops/sec	0.127ms	0.143ms	0.151ms	0.239ms
ZADD	50,000	50	223,214 ops/sec	0.127ms	0.199ms	0.231ms	0.391ms

8.3 Analysis

All seven commands exceed 200,000 operations per second, surpassing the project requirement of 50,000 ops/sec by a factor of four. Latency is consistently sub-millisecond across all command types, with p50 values clustered tightly between 0.127ms and 0.135ms. ZADD has the highest throughput despite being the most complex command. At 223,214 ops/sec, ZADD outperforms even PING. This is explained by the benchmark's access pattern: redis-benchmark reuses a small pool of member names, meaning most ZADD calls are score

updates on existing members rather than true inserts. Score updates remove the old node and reinsert it, but skip the random level generation step, making the common case faster than a cold insert. The p95 and p99 latencies for ZADD (0.199ms and 0.231ms) are the highest among all commands, reflecting the $O(\log n)$ skip list pointer traversal that cold inserts require.

PING is the baseline for raw I/O throughput. At 206,611 ops/sec, PING performs no dictionary access whatsoever — it reads one RESP array, recognizes the command name, and writes +PONG\r\n to the output buffer. Its throughput therefore represents the ceiling imposed by socket I/O and event loop overhead alone, independent of any data structure work.

GET is faster than SET. GET performs a hash table lookup and writes a bulk string reply. SET additionally allocates a new sds string, frees the previous value if the key existed, and may trigger a hash table resize when the load factor crosses 0.75. The resize doubles the bucket array and rehashes all existing entries in a single $O(n)$ pause. This accounts for SET's slightly lower throughput (214,132 vs 208,768 ops/sec) and its higher max latency (0.303ms vs 0.607ms). The Redis solution — incremental rehashing spread across subsequent operations — would eliminate this spike but was judged out of scope.

LPUSH and LPOP are the most consistent commands. Both achieve p99 of exactly 0.151ms with a max of 0.175ms and 0.455ms respectively. Doubly-linked list head and tail operations are $O(1)$ with no memory search — only pointer updates — so execution time is essentially constant regardless of list length. The slightly higher LPOP max latency reflects occasional list cleanup when the list becomes empty and the key is deleted from the top-level dictionary. HSET matches GET in latency despite doing more work. HSET must first look up the top-level key to find the nested HTable, then perform a second hash table operation on the field. Despite this double lookup, p50 and p99 match GET exactly at 0.127ms and 0.151ms. Both lookups hit the same CPU cache lines since the working set fits entirely in L2 cache for the benchmark's key count, making the second lookup essentially free.

Latency distribution is extremely consistent across all commands. The gap between p50 and p99 never exceeds 0.104ms for any command. This consistency is a direct consequence of the single-threaded model: there is no lock contention, no context switching between competing threads, and no cache line invalidation from concurrent writers. Every request sees the same execution environment, producing the tight latency distribution visible in the results.

9. Conclusion

minired demonstrates that the core of a high-performance in-memory key-value store is neither large nor complex. The server is implemented in approximately 2,500 lines of C++17 across fifteen source files. It sustains over 200,000 operations per second with sub-millisecond median latency, passes 100% of its unit tests, and is compatible with an unmodified redis-cli.

The most important lessons from the implementation are architectural rather than algorithmic. The single-threaded event loop is fast not because it does clever things, but because it avoids wasteful ones: no lock acquisition, no context switching, no cache invalidation from competing threads. The RESP parser is correct not because it uses sophisticated parsing techniques, but because it respects the invariant that TCP is a stream and never assumes that one recv equals one message. The skip list is efficient not because its implementation is exotic, but because its expected $O(\log n)$ behavior holds in practice with small constants.

The fork-based persistence mechanism is perhaps the most elegant piece of the design. It provides a consistent point-in-time snapshot of the server's state with zero coordination between the writing process and the serving process, relying entirely on a property of the operating system — copy-on-write virtual memory — that is present on every modern Unix platform. The simplicity of the implementation relative to the correctness guarantee it provides is a reminder that the best systems solutions often involve finding the right abstraction rather than implementing a clever algorithm.